

LINQ Join with Multiple Data Sources in C#

In this article, I will discuss **How to Perform LINQ Inner Join with Multiple Data Sources in C#** with Examples. Before proceeding to this article, I strongly recommend you read our previous article discussing [how to perform LINQ Inner Join with two Data Sources in C#](#) with Examples using Method and Query Syntax.

LINQ Join with Multiple Data Sources in C#

In C#, you can perform a LINQ Join with multiple data sources by using the join keyword and the equals keyword combined with the on keyword. This allows you to combine data from multiple collections or sources based on a common key or condition. Here's how you can do it:

Assuming you have three data sources (collection1, collection2, and collection3) that you want to join and you want to perform an inner join, the basic syntax for a LINQ join with multiple data sources is as follows:

```
var query = from item1 in collection1
            join item2 in collection2 on item1.Key1 equals item2.Key1
            join item3 in collection3 on item1.Key1 equals item3.Key1
            // Add more join clauses as needed
            select new
            {
                PropertyFromCollection1 = item1.Property1,
                PropertyFromCollection2 = item2.Property2,
                PropertyFromCollection3 = item3.Property3,
                // Include properties from other collections as needed
            };
```

In this Example:

- collection1, collection2, and collection3 represent your data sources or collections.
- Key1 is the common key used to perform the joins. You can use any key or condition that matches your data.
- Property1, Property2, and Property3 represent the properties you want to include in the result.

Here's a step-by-step breakdown of the process:

- Start with the first collection (collection1 in this example).
- Use the join keyword followed by the second collection (collection2) and the on keyword to specify the common key (Key1) between the two collections.
- Add additional join clauses to join more collections if needed. Each subsequent join clause specifies the collection to join and the common key.
- In the select clause, create a new object that combines properties from all joined collections. You can include properties from each collection as needed.

- The result of this query will be an IEnumerable of the anonymous type specified in the select clause, containing data from all the joined collections.

Examples to Understand LINQ Join with Multiple Data Sources in C#:

We will use the following three data sources, i.e., Employee, Address, and Department data sources. Here, the AddressId property of the Employee Data Source points to the ID property of the Address Data Source. Similarly, the DepartmentId property of the Employee Data Source refers to the ID property of the Department Data Source. That means there are some common properties between these three data sources, and using those properties, we will perform the Join. The point that you need to remember is to perform the Join Operation, we need to have a common property.

```
ID = 1, Name = "Preety", AddressId = 1, DepartmentId = 10
ID = 2, Name = "Priyanka", AddressId = 2, DepartmentId = 20
ID = 3, Name = "Anurag", AddressId = 3, DepartmentId = 30
ID = 4, Name = "Pranaya", AddressId = 4, DepartmentId = 0
ID = 5, Name = "Hina", AddressId = 5, DepartmentId = 0
ID = 6, Name = "Sambit", AddressId = 6, DepartmentId = 0
ID = 7, Name = "Happy", AddressId = 7, DepartmentId = 0
ID = 8, Name = "Tarun", AddressId = 8, DepartmentId = 0
ID = 9, Name = "Santosh", AddressId = 9, DepartmentId = 10
ID = 10, Name = "Raja", AddressId = 10, DepartmentId = 20
ID = 11, Name = "Ramesh", AddressId = 11, DepartmentId = 30
```

Employee Data Source

Address Data Source

```
ID = 1, AddressLine = "AddressLine1"
ID = 2, AddressLine = "AddressLine2"
ID = 3, AddressLine = "AddressLine3"
ID = 4, AddressLine = "AddressLine4"
ID = 5, AddressLine = "AddressLine5"
ID = 9, AddressLine = "AddressLine9"
ID = 10, AddressLine = "AddressLine10"
ID = 11, AddressLine = "AddressLine11"
```

Department Data Source

```
ID = 10, Name = "IT"
ID = 20, Name = "HR"
ID = 30, Name = "Payroll"
```

Our business requirement is to fetch the following data from the above three data sources. Notice that we are only required to fetch the data that exists in all three data sources. We need the ID and Name property from the Employee Data Source, the Department property from the Department Data Source, and the AddressLine Property value as Address from the Address Data Source.

```
ID = 1, Name = "Preety", Department = "IT", Address = "AddressLine1"
ID = 2, Name = "Priyanka", Department = "HR", Address = "AddressLine2"
ID = 3, Name = "Anurag", Department = "Payroll", Address = "AddressLine3"
ID = 9, Name = "Santosh", Department = "IT", Address = "AddressLine9"
ID = 10, Name = "Raja", Department = "HR", Address = "AddressLine10"
ID = 11, Name = "Ramesh", Department = "Payroll", Address = "AddressLine11"
```

As you can see, we need to fetch only the employees with employee IDs 1, 2, 3, 9, 10, and 11. This is because these 6 employees are present in all the data sources, and while we are performing the LINQ Inner join, it only fetches the records that exist in both data sources.

Creating the Model Classes and Data Sources:

So, create a class file named **Employee.cs** and copy and paste the following code. This class has four properties: ID, Name, AddressId, and DepartmentId. We have also created one method to return a collection of Employees, which will be one of our data sources.

```
using System.Collections.Generic;
namespace LINQJoin
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int AddressId { get; set; }
        public int DepartmentId { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", AddressId = 1, DepartmentId = 1 },
                new Employee { ID = 2, Name = "Priyanka", AddressId = 2, DepartmentId = 2 },
                new Employee { ID = 3, Name = "Anurag", AddressId = 3, DepartmentId = 3 },
                new Employee { ID = 4, Name = "Pranaya", AddressId = 4, DepartmentId = 4 },
                new Employee { ID = 5, Name = "Hina", AddressId = 5, DepartmentId = 5 },
                new Employee { ID = 6, Name = "Sambit", AddressId = 6, DepartmentId = 6 },
                new Employee { ID = 7, Name = "Happy", AddressId = 7, DepartmentId = 7 },
                new Employee { ID = 8, Name = "Tarun", AddressId = 8, DepartmentId = 8 },
                new Employee { ID = 9, Name = "Santosh", AddressId = 9, DepartmentId = 9 },
                new Employee { ID = 10, Name = "Raja", AddressId = 10, DepartmentId = 10 },
                new Employee { ID = 11, Name = "Ramesh", AddressId = 11, DepartmentId = 11 }
            };
        }
    }
}
```

Next, create another class file with the name **Address.cs** and copy and paste the following code. This class has two properties: ID and AddressLine. We have also created one method to return a collection of addresses, which will be the second data source for the Inner Join.

```
using System.Collections.Generic;
namespace LINQJoin
{
    public class Address
    {
        public int ID { get; set; }
        public string AddressLine { get; set; }
        public static List<Address> GetAllAddresses()
        {
            return new List<Address>()
            {
                new Address { ID = 1, AddressLine = "AddressLine1" },
                new Address { ID = 2, AddressLine = "AddressLine2" },
                new Address { ID = 3, AddressLine = "AddressLine3" },
                new Address { ID = 4, AddressLine = "AddressLine4" },
                new Address { ID = 5, AddressLine = "AddressLine5" },
                new Address { ID = 9, AddressLine = "AddressLine9" },
                new Address { ID = 10, AddressLine = "AddressLine10" },
                new Address { ID = 11, AddressLine = "AddressLine11" },
            };
        }
    }
}
```

Finally, create another class file with the name **Department.cs** and then copy and paste the following code into it. This class has 2 properties, i.e., ID and Name. We have also created one method to return a collection of departments, which will be the third data source.

```
using System.Collections.Generic;
namespace LINQJoin
{
    public class Department
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Department> GetAllDepartments()
        {
            return new List<Department>()
            {
                new Department { ID = 10, Name = "IT" },
                new Department { ID = 20, Name = "HR" },
                new Department { ID = 30, Name = "Payroll" },
            };
        }
    }
}
```

Joining three Data Sources using LINQ Query Syntax in C#:

The following code snippet shows how to join three data sources using Linq Query Syntax in C#. This is very much similar to SQL Join. Here, first, we are fetching the employee's collection using the emp variable and then joining it with the addresses collection using the join operator (with the joining condition as **on emp.AddressId equals adrs.ID**). We are accessing the addresses collection using the adrs variable. Then again, we are joining the employee collection with the department collection using the join operator (with the joining condition as **on emp.DepartmentId equals dept.ID**). We are accessing the department collection using the dept variable. Finally, the result is projected into an anonymous type. Here, we are fetching ID and EmployeeName properties from the Employees collection, DepartmentName from the Department collection, and AddressLine from the Address Collection.

```
//Data Source1
from emp in Employee.GetAllEmployees()

//Joining with Address Data Source (Data Source2)
join adrs in Address.GetAllAddresses()
on emp.AddressId equals adrs.ID

//Joining with Department Data Source (Data Source3)
join dept in Department.GetAllDepartments()
on emp.DepartmentId equals dept.ID

//Projecting the result set
select new
{
    ID = emp.ID,
    EmployeeName = emp.Name,
    DepartmentName = dept.Name,
    AddressLine = adrs.AddressLine
}
```

If you want to join the fourth data source, you need to write another join within the query. The complete example code is given below. You can access the query result using a for each loop, which is shown in the example below. The following example code is self-explained, so please go through the comment lines.

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            var JoinMultipleDSUsingQS = (//Data Source1 i.e. Employee
                                         from emp in Employee.GetAllEmployees()
                                         //Joining with Address Data Source (I
Source2)
                                         join adrs in Address.GetAllAddresses()
```

Source3)

```
on emp.AddressId equals adrs.ID
//Joining with Department Data Source

join dept in Department.GetAllDepartments
on emp.DepartmentId equals dept.ID
//Projecting the Result to an Anonymous Type
select new
{
    ID = emp.ID,
    EmployeeName = emp.Name,
    DepartmentName = dept.Name,
    AddressLine = adrs.AddressLine
}).ToList();

//Accessing the Result using a Foreach Loop
foreach (var employee in JoinMultipleDSUsingQS)
{
    Console.WriteLine($"ID = {employee.ID}, Name = {employee.EmployeeName}, Department = {employee.DepartmentName}, Address = {employee.AddressLine}");
}
Console.ReadLine();
}
```

Output:

```
ID = 1, Name = Preety, Department = IT, Address = AddressLine1
ID = 2, Name = Priyanka, Department = HR, Address = AddressLine2
ID = 3, Name = Anurag, Department = Payroll, Address = AddressLine3
ID = 9, Name = Santosh, Department = IT, Address = AddressLine9
ID = 10, Name = Raja, Department = HR, Address = AddressLine10
ID = 11, Name = Ramesh, Department = Payroll, Address = AddressLine11
```

Can we project the result to a named type instead of an anonymous type? Yes, it is possible. Let's see how. First, create a class file with **EmployeeResult.cs** with the required properties you want in the result set. We have created the class with the following four properties per our requirements.

```
namespace LINQJoin
{
    class EmployeeResult
    {
        public int ID { get; set; }
        public string EmployeeName { get; set; }
        public string DepartmentName { get; set; }
        public string AddressLine { get; set; }
    }
}
```

Next, modify the Main method of the Program class as follows. Here, you can see we are projecting the result to the above-created EmployeeResult type.

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            var JoinMultipleDSUsingQS = (//Data Source1 i.e. Employee
                                         from emp in Employee.GetAllEmployees()
                                         //Joining with Address Data Source (ID
Source2)
                                         join adrs in Address.GetAllAddresses()
                                         on emp.AddressId equals adrs.ID
                                         //Joining with Department Data Source (ID
Source3)
                                         join dept in Department.GetAllDepartments()
                                         on emp.DepartmentId equals dept.ID
                                         //Projecting the Result to a Named Type
EmployeeResult
                                         select new EmployeeResult
                                         {
                                             ID = emp.ID,
                                             EmployeeName = emp.Name,
                                             DepartmentName = dept.Name,
                                             AddressLine = adrs.AddressLine
                                         }).ToList();
            //Accessing the Result using a Foreach Loop
            foreach (var employee in JoinMultipleDSUsingQS)
            {
                Console.WriteLine($"ID = {employee.ID}, Name = {employee.EmployeeName},
Department = {employee.DepartmentName}, Address = {employee.AddressLine}");
            }
            Console.ReadLine();
        }
    }
}
```

With the above changes in place, run the application code, and you will also get the same output as the previous example, as shown in the below image.

```
ID = 1, Name = Preety, Department = IT, Address = AddressLine1
ID = 2, Name = Priyanka, Department = HR, Address = AddressLine2
ID = 3, Name = Anurag, Department = Payroll, Address = AddressLine3
ID = 9, Name = Santosh, Department = IT, Address = AddressLine9
ID = 10, Name = Raja, Department = HR, Address = AddressLine10
ID = 11, Name = Ramesh, Department = Payroll, Address = AddressLine11
```

LINQ Method Syntax to Perform Inner Join using Multiple Data Sources in C#:

Let's see how to rewrite the previous example using LINQ Method Syntax to Join three data sources using C#. Writing the join query when joining more than two data sources using LINQ Method Syntax is difficult. The following code snippet shows how to write the query to fetch data from three data sources using the LINQ Method syntax.

```
//Employee data Source (i.e. Data Source 1)
Employee.GetAllEmployees()

//Joining with Address data Source (i.e. Data Source 2)
.Join(
    Address.GetAllAddresses(), //Inner Data Source 1
    empLevel1 => empLevel1.AddressId, //Outer Key selector
    addLevel1 => addLevel1.ID, //Inner Key selector

    //Result set
    (empLevel1, addLevel1) => new { empLevel1, addLevel1 }
)

// Joining with Department Data Source (i.e. data Source 3)
.Join(
    Department.GetAllDepartments(), //Inner Data Source 2

    //You cannot access the outer key selector directly
    //You can only access with the result set created in previous step
    //i.e. using empLevel1 and addLevel1
    empLevel2 => empLevel2.empLevel1.DepartmentId, //Outer Key selector
    deptLevel1 => deptLevel1.ID, //Inner Key selector

    //Result set
    (empLevel2, deptLevel1) => new { empLevel2, deptLevel1 }
)

//Creating the actual result set
.Select(e => new
{
    ID = e.empLevel2.empLevel1.ID,
    EmployeeName = e.empLevel2.empLevel1.Name,
    AddressLine = e.empLevel2.addLevel1.AddressLine,
    DepartmentName = e.deptLevel1.Name
}).ToList();
```

The complete example code is given below. In the example below, we use the LINQ Method Syntax to perform Inner Join between three data sources. The following example code is self-explained, so please go through the comment lines.

```
using System.Linq;
using System;
namespace LINQJoin
{
```

```

class Program
{
    static void Main(string[] args)
    {
        var JoinMultipleDSUsingMS =
            //Employee data Source (i.e. Data Source 1)
            Employee.GetAllEmployees()
            //Joining with Address data Source (i.e. Data Source 2)
            .Join(
                Address.GetAllAddresses(), //Inner Data Source
                empLevel1 => empLevel1.AddressId, //Outer Key selector
                addLevel1 => addLevel1.ID, //Inner Key selector
                //Result set
                (empLevel1, addLevel1) => new { empLevel1, addLevel1 }
            )
            // Joining with Department Data Source (i.e. Data Source 3)
            .Join(
                Department.GetAllDepartments(), //Inner Data Source
                //You cannot access the outer key selector
                //You can only access with the result set
                //i.e. using empLevel1 and addLevel1
                empLevel2 => empLevel2.empLevel1.DepartmentId,
                deptLevel1 => deptLevel1.ID, //Inner Key selector
                //Result set
                (empLevel2, deptLevel1) => new { empLevel2, deptLevel1 }
            )
            //Creating the actual result set
            .Select(e => new
            {
                ID = e.empLevel2.empLevel1.ID,
                EmployeeName = e.empLevel2.empLevel1.Name,
                AddressLine = e.empLevel2.addLevel1.AddressLine,
                DepartmentName = e.deptLevel1.Name
            })
            .ToList();
        foreach (var employee in JoinMultipleDSUsingMS)
        {
            Console.WriteLine($"ID = {employee.ID}, Name = {employee.EmployeeName}, Address = {employee.AddressLine}, Department = {employee.DepartmentName}");
        }
        Console.ReadLine();
    }
}

```

It will give you the same output as the previous example, as shown in the below image.

```
ID = 1, Name = Preety, Department = IT, Address = AddressLine1
ID = 2, Name = Priyanka, Department = HR, Address = AddressLine2
ID = 3, Name = Anurag, Department = Payroll, Address = AddressLine3
ID = 9, Name = Santosh, Department = IT, Address = AddressLine9
ID = 10, Name = Raja, Department = HR, Address = AddressLine10
ID = 11, Name = Ramesh, Department = Payroll, Address = AddressLine11
```

In the above example, we have projected the result to an anonymous type. Instead of projecting the Result to an Anonymous Type, we can use the LINQ Method Syntax to project the result to a named type. We have already created a type called EmployeeResult. Let us project the result to the EmployeeResult type. So, modify the Main method of the Program class as follows.

```
using System.Linq;
using System;
namespace LINQJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            var JoinMultipleDSUsingMS =
                //Employee data Source (i.e. Data Source 1)
                Employee.GetAllEmployees()
                //Joining with Address data Source (i.e. Data Source 2)
                .Join(
                    Address.GetAllAddresses(), //Inner Data Source
                    empLevel1 => empLevel1.AddressId, //Outer Key selector
                    addLevel1 => addLevel1.ID, //Inner Key selector
                    //Result set
                    (empLevel1, addLevel1) => new { empLevel1, addLevel1 }
                )
                // Joining with Department Data Source (i.e. Data Source 3)
                .Join(
                    Department.GetAllDepartments(), //Inner Data Source
                    //You cannot access the outer key selector
                    //You can only access with the result set
                    //i.e. using empLevel1 and addLevel1
                    empLevel2 => empLevel2.empLevel1.DepartmentId,
                    deptLevel1 => deptLevel1.ID, //Inner Key selector
                    //Result set
                    (empLevel2, deptLevel1) => new { empLevel2, deptLevel1 }
                )
                //Creating the actual result set
                .Select(e => new EmployeeResult
                {

```

```

        ID = e.empLevel2.empLevel1.ID,
        EmployeeName = e.empLevel2.empLevel1.Name,
        AddressLine = e.empLevel2.addLevel1.AddressLine,
        DepartmentName = e.deptLevel1.Name
    }).ToList();
    foreach (var employee in JoinMultipleDSUsingMS)
    {
        Console.WriteLine($"ID = {employee.ID}, Name = {employee.EmployeeName}, AddressLine = {employee.AddressLine}, Department = {employee.DepartmentName}");
    }
    Console.ReadLine();
}
}
}

```

Run the application, and you will get the same output as the previous example.

Real-Time Example to Understand LINQ Join Based on 3 Data Sources:

The following example demonstrates how to join three collections: customers, orders, and orderDetails and project the result into a new object that includes the customer's name, order date, and product name. You can choose to use either query syntax or method syntax based on which is more readable or convenient for your specific scenario.

```

using System;
using System.Linq;
namespace LINQDemo
{
    public class Program
    {
        static void Main()
        {
            // Define the data sources
            var customers = new[] {
                new Customer { CustomerId = 1, Name = "Customer A" },
                new Customer { CustomerId = 2, Name = "Customer B" }
            };
            var orders = new[] {
                new Order { OrderId = 1, CustomerId = 1, OrderDate = DateTime.Now },
                new Order { OrderId = 2, CustomerId = 2, OrderDate = DateTime.Now }
            };
            var orderDetails = new[] {
                new OrderDetails { OrderDetailId = 1, OrderId = 1, ProductName = "Product 1" },
                new OrderDetails { OrderDetailId = 2, OrderId = 1, ProductName = "Product 2" },
                new OrderDetails { OrderDetailId = 3, OrderId = 2, ProductName = "Product 3" },
                new OrderDetails { OrderDetailId = 4, OrderId = 2, ProductName = "Product 4" }
            };
            // Perform the join using query syntax
            var querySyntax = from customer in customers

```

```

order.CustomerId
detail.OrderId

join order in orders on customer.CustomerId equals
join detail in orderDetails on order.OrderId equals
select new
{
    customer.Name,
    order.OrderDate,
    detail.ProductName
};
// Perform the join using method syntax
var methodSyntax = customers
    .Join(orders,
        customer => customer.CustomerId,
        order => order.CustomerId,
        (customer, order) => new { customer, order })
    .Join(orderDetails,
        co => co.order.OrderId,
        detail => detail.OrderId,
        (co, detail) => new {
            co.customer.Name,
            co.order.OrderDate,
            detail.ProductName
        });
// Execute the query and print the results
foreach (var result in querySyntax) // or methodSyntax
{
    Console.WriteLine($"{result.Name} placed an order on {result.OrderDate} for {result.ProductName}");
}
Console.ReadKey();
}
}

public class Order
{
    public int OrderId { get; set; }
    public DateTime OrderDate { get; set; }
    public int CustomerId { get; set; }
}

public class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
}

public class OrderDetails
{
    public int OrderDetailId { get; set; }
    public int OrderId { get; set; }
    public string ProductName { get; set; }
}
}

```

You can extend this pattern to join multiple data sources as needed, as long as you have common keys or conditions to perform the joins.

My Personal Opinion: If you want to fetch data from two data sources, you can use either the Method syntax or Query Syntax. However, if you want to fetch data from more than two data sources, it is easier to use query syntax than method syntax.

In the next article, I will discuss the [LINQ Group Join in C#](#) with Examples. In this article, I will try to explain how to perform a LINQ inner join with multiple data sources using C# with examples. I hope you understand joining multiple data sources using LINQ in C# with Examples.